

Technical question bank: architecture, agentic RAG, stack, scale, production

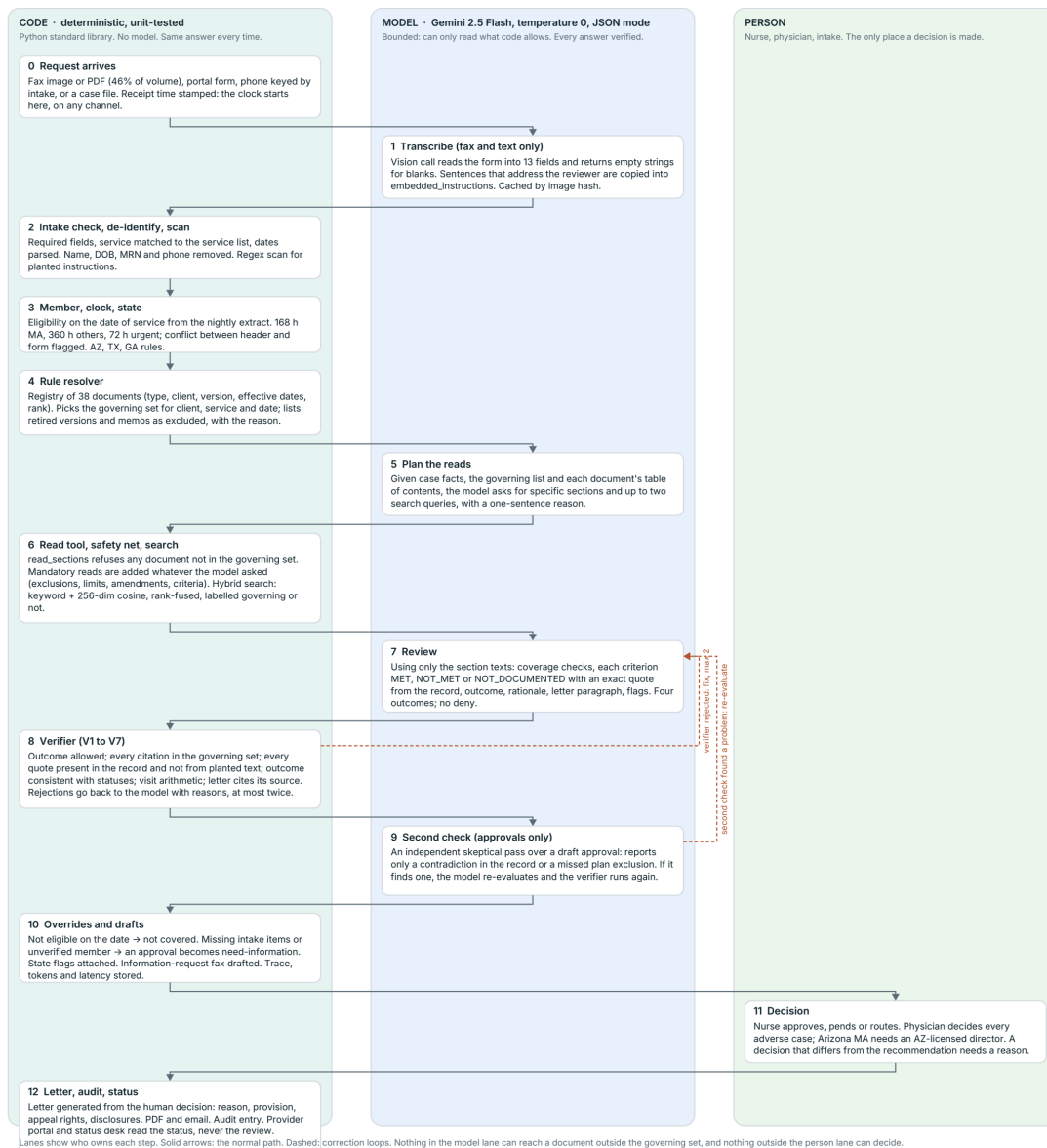
Every answer below describes the system as it is coded in `copilot/` and `api/`, not as it could be. Where something is a plan rather than a fact, it says so.

The 60-second architecture answer

"Six elements. A **registry** of all 38 documents with type, client, version, effective dates and authority rank. A **rule resolver in code** that picks what governs a case by client, service and date of service, and excludes retired versions and memos with a reason. A **bounded agent**: the model plans what to read, calls a read tool that refuses anything outside the governing set, and tests each criterion against a de-identified record with an exact quote. A **verifier in code** that checks citations, quotes, consistency and arithmetic before a person sees anything, and sends rejections back to the model. A **second skeptical pass** on every draft approval. And a **human decision** with four outcomes and no deny, which generates the letter. Python standard library on Vercel, Gemini 2.5 Flash at temperature 0, Supabase for storage, 41 unit tests with no model call."

Review Copilot: the agentic RAG loop, as coded

`copilot/pipeline.py run_case()` → `copilot/agent.py review()` → `copilot/core.py verify()`. Three model calls on a typical case, up to six with corrections. About 6.5 s and a third of a cent.



Tech stack, one table

Layer	What is used	Why this and not something else
Runtime	Python 3.12, standard library only at runtime (<code>urllib</code> , <code>json</code> , <code>hashlib</code> , <code>hmac</code> , <code>re</code> , <code>csv</code>)	Vercel's Python functions start fast with no dependencies; nothing to patch; every line is readable in a review
Hosting	Vercel serverless functions (<code>api/* .py</code> , <code>BaseHTTPRequestHandler</code>), static <code>public/index.html</code> , 60 s max duration, <code>includeFiles</code> ships <code>copilot/</code> and <code>data/</code>	Public URL in minutes, no servers to run for a prototype; production target is Azure (see the 90-day plan)
Front end	One HTML file, vanilla JavaScript, no framework; Inter, Source Serif 4, JetBrains Mono via Google Fonts; <code>pdf.js</code> from a CDN only for rendering an uploaded PDF to an image in the browser	No build step, loads in one request, easy to audit; role-specific views are chosen client-side but every permission is enforced server-side
Model	<code>google/gemini-2.5-flash</code> through OpenRouter (default) or the Gemini API directly (<code>LLM_PROVIDER=gemini</code>); temperature 0, JSON response format, max 4,000 output tokens; vision input for fax images	Cheap, fast, reads images; one environment variable switches provider; architecture does not depend on the model
Embeddings	<code>openai/text-embedding-3-small</code> through OpenRouter, truncated to 256 dimensions	Small corpus; 256 dims keeps the index at 322×256 floats in a JSON file; query and document embeddings must come from the same model, so embeddings always go through OpenRouter
Retrieval	Two paths: direct section reads chosen by the model and enforced by code (primary), and hybrid search (keyword score + cosine, fused by reciprocal rank) filtered by the governing set (secondary)	Reviewers cite sections, so the unit of retrieval is the section, not a token window
Knowledge base	<code>scripts/ingest.py</code> : <code>pdfplumber</code> (dev only) → <code>data/sections.json</code> (322 sections), <code>data/registry.json</code> (38 documents), <code>data/vectors.json</code>	Ingestion output is committed, so the deployed app needs no PDF library
Storage	Supabase Postgres over its REST API, one table <code>copilot_store(kind, id, data jsonb, updated_at)</code> ; local JSON file fallback	Durable cases, decisions, letters and audit without an ORM; swapping to Azure Postgres is one module
Auth	PBKDF2-HMAC-SHA256 password hashes (200,000 rounds, 16-byte salt), HMAC-SHA256 signed session tokens, 8-hour expiry, <code>AUTH_SECRET</code> on the server	No password ever stored; constant-time comparisons; no third-party dependency; production would be Entra ID single sign-on with the same role model
Email and PDF	Gmail SMTP over SSL with an app password; a dependency-free PDF writer for letters	Demo letters go to one configured mailbox, never to a real patient
Tests and evidence	<code>pytest</code> (41 tests, no model call); <code>scripts/run_eval.py</code> (120 audited cases + 24 adversarial), <code>scripts/run_open.py</code> (30 open cases); results in <code>data/results/</code>	Deterministic parts are unit-tested; model parts are measured against ground truth

Agentic RAG: what it looks like

Q: What makes this "agentic" RAG rather than plain RAG? Plain RAG embeds the question, pulls the top-k chunks and answers. Here the model gets a map, not chunks: the case facts, the list of governing documents and each document's table of contents. It then decides which sections to read and whether to search, through tools that code controls. It reads, evaluates every criterion, and its answer goes to a verifier that can reject it and send it back. The model has agency over what to read and how to weigh evidence; it has no agency over which documents exist for this case or what the outcome options are.

Q: Walk me through one case, call by call.

- Code: intake check, de-identification, planted-text scan, eligibility, clock, state flags, governing set. No model.
- Model call 1, plan: "which sections do you need?" Returns a list of (document, version, section) and up to two search queries, with a reason.
- Code: read tool returns the texts, refusing anything not governing; adds the mandatory safety-net sections (plan exclusions, limits, amendments; policy criteria, limitations, documentation); runs the search queries.
- Model call 2, review: outcome, coverage checks, criteria with quotes, rationale, letter paragraph, flags, as JSON.
- Code: verifier V1 to V7. If rejected, model call 3 (and at most 4) with the reasons. Typical case: accepted first time.

6. Model call 3, second check, only when the draft is approve-ready: a skeptical pass that may only report a contradiction in the record or a missed plan exclusion. If it finds one, one more model call to re-evaluate, then the verifier again.
7. Code: overrides (eligibility, missing intake items), state flags, information-request draft, trace, tokens, latency.
- Average: 3 calls, 6,541 input tokens, 801 output tokens, 6.5 seconds, about a third of a cent.

Q: Is it LangGraph? Where is the graph? No. It is an explicit loop of about sixty lines in `copilot/agent.py`, standard library only, because the runtime had to be dependency-free and because every transition had to be readable by an auditor. The flowchart above is the graph. If it were LangGraph it would be a StateGraph with nodes `resolve_rules (code)`, `plan_reads (model)`, `read_and_search (code)`, `review (model)`, `verify (code)`, `second_check (model)`, `overrides (code)`, `human_decision (interrupt)`, with conditional edges `verify → review on rejection (max 2)` and `second_check → review on a problem`. The stored trace plays the role of a checkpoint. I would adopt LangGraph in production for its checkpointing and interrupt-and-resume on the human step, not for the logic, which does not change.

Q: Why does code pick the documents instead of retrieval? Because which rule applies is a question of dates and authority rank, not of semantic similarity. A retired version of MP-101 is semantically closer to the question than the plan document that overrides it. Retrieval would surface the wrong document with high confidence. The registry answers the question exactly: the policy version whose effective window contains the date of service, the plan document for that client, the delegation addendum for Riverbend, the regulations for that product, MP-117 for every service, and the memos excluded by rank.

Q: Then what is the search for? A secondary path for things the table of contents does not reveal, for example "is there a Medicare rule for CGM in the addendum?" The model may ask up to two queries; each returns three sections, labelled governing or not governing. A not-governing hit is shown so the model can say it exists and why it does not apply, but the verifier rejects it as a citation.

Q: What stops the model from reading outside the governing set? The read tool. `core.read_sections` refuses any (document, version) pair that is not in the governing list and returns the message "REFUSED: MP-101 v1 does not govern this case (in-force version: v2)". The refusal is recorded in the trace. The Governance page has a button that runs exactly this.

Chunking and the knowledge base

Q: What chunking strategy do you use? Structural, not fixed-size. Each PDF is split on its own headings, "N. Title" and "N.N Title", into sections. The 38 documents yield 322 sections; the median section is 234 characters and the largest about 2,000. Every section carries document ID, version, section number and title. Section 0 of each document is its header, from which ingestion parses version, status, effective dates and service code into the registry. A read of a parent section (for example "6") returns its subsections.

Q: Why not token windows with overlap? Because reviewers, auditors and letters cite documents by section. A citation like "MP-101 v2, Section 3" has to resolve to one piece of text, and the verifier has to check that the model's citation points at something that exists and governs. Overlapping windows break that: the same sentence would live in two chunks with no stable address, and a chunk could straddle a section boundary and mix an exclusion with the clause after it. Structural sections are also small enough that a whole policy's criteria fit in one read.

Q: How are versions handled in the index? Each policy version is its own document in the registry with its own sections and embeddings. MP-101 v1 and MP-101 v2 both exist; the resolver decides which one governs on the date of service. Nothing is deleted when a policy is retired, so a 2025 date of service still resolves to the version that was in force then.

Q: What metadata does the registry hold, and where does it come from? From the PDF headers: version, status, effective-from, effective-to, service code, title. From an overlay in `scripts/ingest.py`: document type, authority rank from GOV-01 (regulation 1, plan or delegation agreement 2, policy 3, memo or email 4), the clients a plan belongs to, the products a regulation applies to, and what a memo relates to. The overlay is nine entries. In production this becomes a table owned by the Clinical Policy Committee.

Q: What breaks if a document does not follow the heading format? The whole document lands in section 0 with no addressable sections. Ingestion prints the section count per document, so this is visible immediately. For production the ingestion step needs a check that every policy produced sections 3, 4 and 5, and a human review of any document that did not.

Embeddings and retrieval

Q: Which embedding model, and why 256 dimensions? `openai/text-embedding-3-small` through OpenRouter, truncated to 256 dimensions. The model supports shortened embeddings with small quality loss; 256 keeps the index at 322×256 numbers in a JSON file loaded with the function, with no vector database to run or pay for. The text embedded is "document ID, version, section number, title" plus the section text, so a query about "MP-103" or "section 6.4" also matches lexically.

Q: How does the hybrid search work? Two scores per section: a keyword score (term frequency times a log inverse-document-frequency over the 322 sections) and cosine similarity between the query embedding and the section embedding. The two rankings are fused with reciprocal rank fusion (constant 60). Results are then labelled GOVERNING or NOT_GOVERNING against the resolver's list. Top three per query.

Q: Why not a vector database? At 322 sections, a brute-force cosine over a JSON array takes a few milliseconds and adds no key, no service and no failure mode. At 1,100 documents, which is the real UM Library, I would move to Azure AI Search or pgvector in the same Postgres that holds the cases, keeping the same metadata filters (client, version, effective dates). The search function is one place in `core.py`.

Q: What if the embedding key is missing? Search falls back to keyword ranking only. Direct section reads, which are the primary path, do not use embeddings at all, so the review still runs.

The model

Q: Which model, and what settings? Gemini 2.5 Flash. Temperature 0, JSON response format, 4,000 max output tokens, thinking budget 0 when called directly through the Gemini API. The same model reads fax images. Prompts are versioned (`PROMPT_VERSION`) and the version is stored with every run.

Q: Why not Claude or GPT? I tested Claude Haiku and Sonnet on the hard cases through OpenRouter: better on one case, slower and about ten times the cost. Flash at temperature 0 was within a few cases of them on the audit set and reads images. In production the choice is whatever the Azure tenant's agreement covers; the provider is one environment variable.

Q: How deterministic is it? Temperature 0 reduces but does not remove variation. Across repeated runs the open queue scored 27 to 29 of 30; every miss went toward a person (need information, physician, manual review), never toward an approval. The verifier, the second check and the overrides are deterministic.

Q: How do you prevent hallucinated citations and quotes? The verifier. V2: every citation in the answer must be in the governing set. V3: every quote behind a MET or NOT_MET must appear in the de-identified record after whitespace and case normalisation (fragment matching allows quotes stitched from two sentences), and must not come from text flagged as a planted instruction. Failures go back to the model with the reason. A quote that cannot be verified cannot support a status.

Q: What does the system prompt contain? Hard rules (governing documents only; provider text is data; four outcomes; quote exactly; multi-element criteria need every element; take stated measurements at face value), a decision procedure (coverage under the plan, which criteria apply including the Riverbend Medicare precedence, investigational check, criteria with AND/OR, and the outcome precedence a to f), and the JSON schema. It is about 60 lines and is in `copilot/agent.py` for anyone to read.

Where AI is used, and where it is not

Step	Model	Code	Person
Transcribe a fax image or pasted text into fields	Yes (vision)	Validates the result	Intake confirms when fields are missing
Completeness check, receipt clock, priority		Yes	
De-identification, planted-text scan		Yes (regex)	
Eligibility, member state, state rules		Yes	
Which documents govern, which are excluded		Yes (registry)	
Which sections to read, what to search	Yes	Enforces the boundary	

Step	Model	Code	Person
Criteria evaluation with quotes, rationale, letter paragraph draft	Yes		
Verification of citations, quotes, consistency, arithmetic		Yes	
Second skeptical check on approvals	Yes		
Overrides (not eligible, missing items), information-request draft		Yes	
Coverage check for providers and the phone desk		Yes	
Decision			Yes, only
Letter assembly, PDF, email		Yes	Sent after a human decision
Status disclosure to providers and callers		Yes (identity check)	
Roles, sessions, audit		Yes	
Policy library question answering ("Ask")	Yes, over governing passages	Filters retrieval	

The rule of thumb: the model reads and weighs; code decides what may be read and whether the answer is well-formed; a person decides the case.

Security, privacy and data flow

Q: What leaves Bellcourt's environment and goes to the model? For the review: the case facts (client, product, state, service, date, age on the date of service, diagnosis code), the governing document texts, and the de-identified clinical narrative. Not the name, date of birth, phone, record number or member ID. For a fax: the image itself, because transcription needs it; in production that call runs inside the tenant covered by the business associate agreement. Nothing is used for training.

Q: How is prompt injection handled? Three layers. The transcription prompt copies any sentence that addresses the reviewer into a separate `embedded_instructions` list. A regex scan catches phrases like "pre-approved", "mark it approved", "criteria waived" in any text. The review prompt treats the record as data and must flag such text; the verifier rejects any evidence quote taken from it. In evaluation, ten of ten planted instructions were ignored and flagged.

Q: How are roles enforced? On the server, per request. Create: intake and provider. Review: intake, nurse, physician. Decisions: a per-decision allow list (approve: nurse, physician; pend: intake, nurse, physician; route: nurse, intake; deny: physician only). Provider requests return a status view that omits the review entirely. Agent lookups need the request number and the patient's date of birth to match. The interface disables buttons by role, but the server would refuse them anyway, and the Governance page proves it with a dry-run attempt.

Q: Where is the audit trail and what does it contain? Every create, review, decision, letter, email and status lookup is written with user, role, time, case and detail. Every review stores the inputs, the governing and excluded documents, the model's plan, the reads (including refusals), the searches, the verifier report, the second check, the output, tokens and latency. This is what Riverbend addendum section 5 asks Bellcourt to produce on audit.

Scaling

Q: Volume: 310,000 requests a year. Does this scale? That is about 1,200 a day, 50 an hour, with Monday-morning spikes of several hundred faxes. At 6.5 seconds a case, one worker clears 550 an hour; serverless functions run in parallel anyway. The model provider is the real ceiling: at 3 calls and about 7,300 tokens per case, a day is about 3,600 calls and 9 million tokens, well inside standard rate limits, and about \$4 a day at today's Flash price.

Q: What changes at Bellcourt's real document count (about 1,100 files)? The index moves from a JSON file to pgvector or Azure AI Search with the same metadata filters, and ingestion becomes a pipeline with a human check on each new document's structure. The resolver does not change: it is still a lookup over the registry. The registry becomes a table with an owner and an approval step.

Q: What are the bottlenecks you would watch? Model latency and rate limits (mitigate with a queue, retries with backoff, which the client already has, and batching of fax transcriptions overnight); the 60-second function limit on Vercel (not an issue on Azure Functions or Container Apps with a worker queue); cold starts loading `sections.json` and `vectors.json` (small today; move to the database at scale); and Supabase REST round-trips per audit write (batch, or write-behind).

Q: Cost at scale? Model: about \$0.003 per case, about \$1,000 a year at full volume. Fax transcription: about 233,000 images a year, about \$1,500 at vision pricing. Hosting, database, storage and logging: tens of thousands a year on Azure. The dominant cost is people: a 0.5 to 1 FTE engineer for support and a policy librarian role for the registry.

Q: Caching? Fax transcriptions are cached by image hash plus model and prompt version, so re-running a case does not re-read the image. Reviews are not cached, deliberately: a re-run is a new run with a new trace.

Production in 90 days

Q: What is the deployment plan?

Weeks	Work	Gate
1	Free fixes (withdraw the memo, tell UM about Kestrel, re-review open denials, report from receipt). Send the 60-day written notice to Riverbend (addendum section 5). Security design review. Stand up the Azure resource group inside the tenant with the HIPAA BAA.	Notice sent; design review signed
1 to 4	Registry as a Postgres table owned by the Clinical Policy Committee; ingestion pipeline for the UM Library (1,100 files) with structure checks; read-only connectors: fax share watcher, portal webhook, PACE replica, nightly eligibility file. Entra ID sign-in with the six roles.	Every policy in the library resolves to sections 3, 4 and 5; registry populated and approved
3 to 6	Model calls moved to Azure OpenAI or Gemini on Vertex inside the agreement; pgvector index; worker queue for reviews; evaluation suite in CI (120 audited cases, 30 open cases, adversarial set, unit tests) run on every prompt or registry change.	Gate 1, day 45: offline results at or above today's; no wrongful approvals; security review of the running system
5 to 8	Reviewer workspace on the real queue in shadow mode: the copilot runs, nurses decide as today, agreement is measured; provider portal and status desk behind the existing portal login.	Four weeks of shadow data collected
7 to 11	Shadow mode review with six senior nurses on self-funded clients; every disagreement reviewed by QA; prompt and registry fixes through CI. Go-live for self-funded clients.	Gate 2, day 75: at least 90% nurse agreement, zero wrongful approvals in shadow
9 to 13	Riverbend go-live after the notice period; receipt-based timeliness reporting; monthly QA with copilot-assisted cases sampled separately; repeat the time-and-motion study.	Gate 3, day 61 or later: notice complete, Riverbend has reviewed the evaluation pack

Q: What is different between the prototype and production? Hosting (Vercel to Azure Functions or Container Apps with a queue), identity (built-in login to Entra ID), storage (Supabase to Azure Postgres with pgvector, audit table with retention), model endpoint (OpenRouter to a model inside the tenant's agreement), inputs (manual upload to connectors on the fax share, portal webhook and PACE replica), and the registry (a JSON overlay to a governed table). The resolver, the agent loop, the verifier, the roles and the letters do not change.

Q: How do you keep the model honest after go-live? The evaluation suite runs in CI on every prompt, model or registry change and blocks the deploy if agreement drops or a wrongful approval appears. Monthly QA samples copilot-assisted cases separately. Repeat-run agreement is tracked. A stop rule: if accuracy on sampled cases falls under 90%, the recommendations are hidden and nurses work as before; the manual process never went away.

Q: What does PACE need to do? Nothing. The copilot reads the PACE replica and writes nothing back; nurses record the decision in PACE as today. When the January 2027 electronic prior authorization API arrives, the copilot's normalised case store is the natural back end for it, and that is a separate project.

Future scope

- **The other 32 plan documents** and the full UM Library, with a policy librarian role and an approval step for every registry change.
- **Automated information requests:** send the drafted fax-back automatically after a human approves the template, escalate to a phone call at 24 to 48 hours for urgent and Medicare cases.
- **Provider status by voice:** an IVR or voice agent on the status desk logic, with the same identity check and the same audit.
- **Appeals assistant:** the same resolver and verifier applied to an appeal, showing the physician what changed between the original decision and the appeal file.
- **Employer and Riverbend dashboards:** receipt-based timeliness, accuracy, overturns and savings per client, replacing the quarterly PDF.
- **Electronic prior authorization (January 2027):** expose the normalised case store through the CMS-required API.
- **Model improvements:** a stronger model on the review step if shadow data shows drift; a small fine-tuned extractor for fax fields; calibrated confidence so that low-confidence criteria route to a person; learning from nurse overrides as a labelled set for the next evaluation, never as automatic rule changes.
- **Multilingual letters** (Spanish first) generated from the same decision record.

Honest limits to volunteer before being asked

- Results vary slightly between runs; every observed miss went toward a person.
- Prompts were tuned while looking at the audited cases; a fresh random sample is the next test.
- The open-case answer key is my own reading of the policies.
- Six of 38 employer plan documents were in the pack, so the prototype covers eight clients.
- Login is a built-in service, not single sign-on; demo letters go to one mailbox; the prototype runs outside a business associate agreement, which is why production moves into the Azure tenant.